

INFOSYS 720 – Information Systems Research
Assignment 3: Theorise Information Systems Artefacts

Aidil Muslim
463734221

Title:

**Task-Aware Prompt Calibration for Generating Synthetic Novice
Python Code**

Paper Template:

Association for Computing Machinery (ACM) - SIG Proceedings
Template

Task-Aware Prompt Calibration for Generating Synthetic Novice Python Code

Aidil Muslim
amus790@aucklanduni.ac.nz
University of Auckland
Auckland, New Zealand

ABSTRACT

Large language models offer a potential source of synthetic code for evaluating educational analysis and feedback systems. However, synthetic code may not reproduce the stylistic and structural defects found in authentic novice submissions. This study proposes a task-aware, iterative prompt-calibration approach for generating functionally correct Python code with realistic novice-style defects. The research suggests three conceptual artefacts that classify observable defect characteristics according to task dependence, build context-aware target profiles, and iteratively calibrate the synthetic generation against empirical defect thresholds. The defect classifications are derived based on the authentic first-year computer science (CS1) submissions. The frameworks distinguish functional correctness from representational similarity while using observed discrepancies to refine generation constraints. Ultimately, the artefacts offer a reusable conceptual basis for producing synthetic novice code that is more systematically faithful in representing authentic student-code characteristics.

CCS Concepts

• Social and professional topics~ Professional topics~ Computing education~ Computing education programs~ Computer science education~ CS1

Keywords

novice programming; code style defects; synthetic data; AI code generation; prompt engineering

1. Theoretical Foundation and Extant Artefacts

The Representation Theory grounds the kernel theory for this research. It proposes that an information system represents selected phenomena from a real-world domain, enabling those phenomena to be reasoned about through an information-system representation (Recker et al., 2019; Wand & Weber, 1988). The usefulness of such a representation depends partly on how faithfully it corresponds to the relevant real-world phenomena. In this study, authentic computer science first-year (CS1) Python submissions constitute the real-world domain, while observable code-quality defects in functionally correct novice programs constitute the phenomena of interest. Empirical defect profiles represent these characteristics conceptually, while generated Python submissions form the synthetic representation. Thus,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee, provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

INFOSYS720, The University of Auckland

© 2026 Copyright held by the owner/author(s)

representation theory emphasises the importance of correspondence between authentic and synthetic.

However, the theory does not specify which novice-code characteristics should be represented or how synthetic representations should be generated and assessed. These aspects are informed by extant artefacts and evidence identified through the systematic literature review (SLR). The SLR shows that novice programs exhibit recurring errors, style anomalies, structural characteristics, and semantic patterns beyond functional correctness (Denzler et al., 2024; Fwa, 2024; Haindl & Weinberger, 2024; Zhang et al., 2024). Within this evidence base, the defect catalogue developed by Rechackova et al. (2024, 2025) provides a conceptual basis for defining what should be represented, while Leinonen et al. (2025) provide a process-oriented precedent for generating synthetic student code and comparing it with authentic data.

1.1 Novice Code-Quality Defect Catalogue

The SLR indicates that novice code contains observable characteristics beyond functional correctness, including errors, style anomalies, structural features, and semantic patterns (Denzler et al., 2024; Fwa, 2024; Haindl & Weinberger, 2024; Zhang et al., 2024). This evidence supports the need for a structured representation of such characteristics. Against this empirical background, Rechackova et al. (2024) proposed an organisational framework and catalogue containing 80 code-quality defects, with particular attention to novice programs. Their subsequent study further organised the defects according to defect types and associated programming constructs, supported by educator assessment, automated detection, and empirical prevalence analysis (Rechackova et al., 2025)

For this research, the catalogue provides a structured vocabulary for specifying which novice-code characteristics should be represented. It also supports the distinction between functional correctness and code quality, since a program may satisfy task requirements while still containing stylistic, structural, or maintainability-related defects. However, the catalogue primarily establishes which defects exist and how they are organised. It does not represent how defect occurrence varies across programming tasks or distinguish broadly recurring tendencies from task-dependent ones. An additional task-aware representation is therefore required to model these occurrence patterns across authentic CS1 submissions.

1.2 Synthetic Buggy-Code Generation and Distributional Evaluation

Leinonen et al. (2025) provide an extant process artefact more directly concerned with synthetic representation. They used an LLM to generate synthetic buggy submissions for introductory programming exercises and compared distributions of test-case failures in generated and authentic student submissions. In Representation Theory terms, authentic submissions form the

focal domain, test-case failure patterns are the represented phenomena, and generated programs are synthetic representations. Correspondence is assessed at the distribution level rather than through the apparent realism of individual examples.

This principle is important because a few plausible generated programs do not demonstrate that a synthetic dataset represents broader novice tendencies; however, Leinonen et al. primarily model functional failure patterns.

1.3 Implications for the Proposed Artefacts

The two extant artefacts provide complementary foundations. Rechackova et al. define and organise the novice-code characteristics to be represented, while Leinonen et al. demonstrate generation and distribution-level comparison with authentic student data. However, neither develops task-aware defect profiles nor iteratively calibrates generation against those profiles. These limitations motivate three proposed conceptual artefacts: a context-aware characteristic classification framework, a context-aware synthetic representation framework, and an iteratively calibrated synthetic generation framework.

2. Proposed Conceptual Artefacts

The proposed conceptual artefacts are organised under the Domain-Informed Synthetic Representation Model (DISRM), which provides the high-level conceptual overview of the research. DISRM describes how empirical observations from a real-world domain are abstracted into contextual profiles, which then constrain the generation of synthetic representations and inform their use in downstream domain applications. The model also recognises that authentic domain observations can directly ground application development, while application use may provide further observations that support continued refinement. In this study, the general model is instantiated in the context of synthetic novice Python code, where authentic CS1 submissions provide the empirical domain and the synthetic generation is guided by the defect classifications based on the domain data.

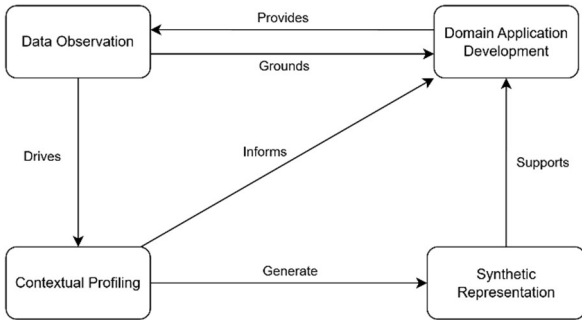


Figure 1. Domain-Informed Synthetic Representation Model (DISRM)

Building on Representation Theory and the extant artefacts discussed in Section 1, this research proposes three complementary conceptual artefacts. The first defines context-aware characteristic classification for representing observable novice-code characteristics. The second generalises these classifications into a Context-Aware Synthetic Representation Framework that separates characteristics according to whether their occurrence is relatively stable across contexts or varies with context. The third operationalises this representation through an Iteratively Calibrated Synthetic Generation Framework for generating and refining synthetic code. Together, the artefacts

progress from defining the phenomena to be represented, modelling their contextual structure, and specifying how that representation can guide synthetic generation.

2.1 Context-Aware Characteristic Classification Framework

The first artefact establishes the conceptual vocabulary required to represent novice code-quality characteristics consistently. This addresses the issue identified in the systematic review that novice-code characteristics are represented in different ways and that a limited distinction is made between characteristics that occur broadly and those associated with particular programming tasks. A novice code-quality defect is defined in this study as an observable stylistic, structural, or maintainability-related characteristic of novice program code that may occur even when the program satisfies its functional requirements (Effenberger & Pelanek, 2022; Rechackova et al., 2024, 2025). The framework focuses on observable properties of submitted code. This boundary is consistent with the intended representational scope of the research, in which authentic submissions and their measurable characteristics constitute the phenomena to be represented.

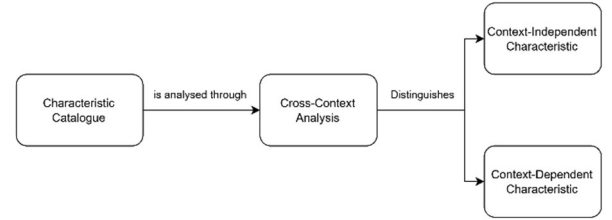


Figure 2. Context-Aware Characteristic Classification Framework

A task-independent defect tendency refers to a defect whose occurrence remains comparatively consistent across different programming tasks. Such a tendency represents a more general characteristic of the observed novice-code population rather than one strongly associated with the requirements of a particular task. In contrast, a task-dependent defect tendency refers to a defect whose occurrence varies according to the programming task or task context. For example, defects associated with iteration, branching, validation, or decomposition may only become relevant when the corresponding programming classifications are required or encouraged by the task.

A defect profile represents the observed occurrence of a selected collection of defect categories within a defined set of submissions. Rather than treating individual defects in isolation, the profile provides a distributional representation of multiple characteristics. A target defect profile is the empirical profile derived from authentic submissions and selected as the representational target for subsequent synthetic generation.

These classifications provide the conceptual basis for RQ1 by making the distinction between task-independent and task-dependent defect tendencies explicit. They also establish the terminology required by the subsequent framework and calibration framework. In particular, the classifications address the requirement to classify defects according to cross-task occurrence and to construct empirical task-aware profiles rather than relying on a single aggregate description of novice code.

2.2 Context-Aware Synthetic Representation Framework

The second artefact is a Context-Aware Synthetic Representation Framework. The framework generalises the task-aware distinction beyond the immediate CS1 setting by proposing that

synthetic generation should represent both characteristics that remain comparatively stable across contexts and characteristics whose occurrence varies according to context.

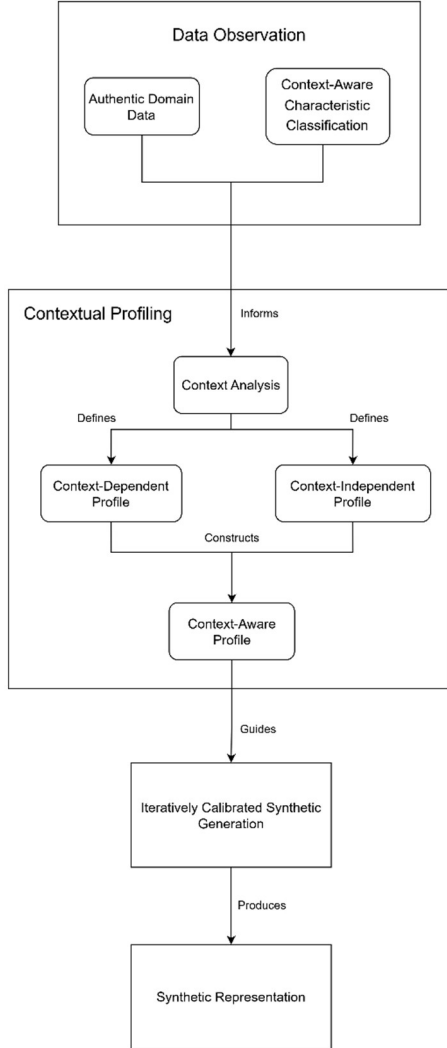


Figure 3. Context-Aware Synthetic Representation Framework

The framework, as shown in Figure 2, begins with authentic domain data, from which relevant observable characteristics are identified. These characteristics are then examined according to their variation across relevant contexts. The resulting empirical representation is separated into two components, a context-independent profile and a context-dependent profile. The two components are combined to form a context-aware target profile, which subsequently guides the production of a synthetic representation.

The distinction between the two profile components is important because an aggregate empirical profile may conceal meaningful contextual variation. A characteristic that appears infrequently across an entire dataset may be common within a specific context. In contrast, a characteristic that appears dominant in aggregate data may largely reflect contexts with greater numbers of observations. Thus, the framework utilises contextual variation as part of the profile representation rather than as noise.

Within the present research, the general framework is instantiated with authentic CS1 Python submissions as the domain data, code-quality defects as the observable

characteristics, and programming tasks as the relevant context. The context-independent component, therefore, corresponds to task-independent defect tendencies, while the context-dependent component corresponds to task-dependent defect tendencies. Their combination produces a task-aware target defect profile for the programming task being represented. Synthetic novice Python submissions then form the resulting synthetic representation.

This framework primarily addresses RQ1 by providing a structure for distinguishing cross-task and task-specific tendencies, while also supporting RQ2 by defining the target representation that can be incorporated into generation. Its broader contribution is that the representational logic is not inherently restricted to Python code. The same structure could, in principle, be applied to other forms of synthetic data where observable characteristics contain both broadly recurring and context-sensitive patterns. The domain, characteristics, and meaning of context may change, while the representational structure remains applicable.

2.3 Iteratively Calibrated Synthetic Generation Framework

The third artefact is an Iteratively Calibrated Synthetic Generation Framework. Whereas the preceding framework describes how empirical characteristics are represented, the framework specifies how the resulting target profile can guide and progressively refine LLM-based generation.

The framework begins with the context-aware target profile produced by the framework. Relevant characteristics from this profile are translated into generation constraints that communicate which characteristics should be represented in the generated output. In the present study, these constraints are incorporated into an LLM prompt together with the original programming task requirements. The resulting prompt, therefore, contains two distinct forms of guidance: fixed requirements that preserve the intended programming task and adaptive representational constraints derived from the target defect profile.

Subsequently, the LLM generates a set of candidate synthetic submissions. The generated programs are subjected to functional validation to determine whether they satisfy the original programming task. This stage separates functional validity from representational similarity. A program that exhibits the intended characteristics but fails to solve the task is not considered an appropriate synthetic representation for this research.

Functionally valid programs undergo defect analysis. It checks which targeted characteristics are detected and aggregated into a synthetic defect profile. This observed profile is compared with the threshold profile. This comparison is meant to identify representational discrepancies, which may disclose underrepresented or overrepresented categories, and target characteristics that are absent from the synthetic dataset.

The identified discrepancies provide the basis for prompt calibration. The relevant generation constraints are adjusted to increase or reduce the emphasis on particular characteristics. A new generation cycle is then performed and evaluated until it satisfies the functionality validation and representation similarity targets.

This framework addresses the second major research problem identified in the previous phase: the lack of an integrated method for translating empirical defect profiles into iterative, task-aware LLM prompts while preserving both functional correctness and alignment with authentic defect distributions. It directly supports RQ2 by defining the profile-to-prompt transformation and RQ3 by providing the mechanism through which iterative refinement can be compared with a non-adaptive prompting baseline.

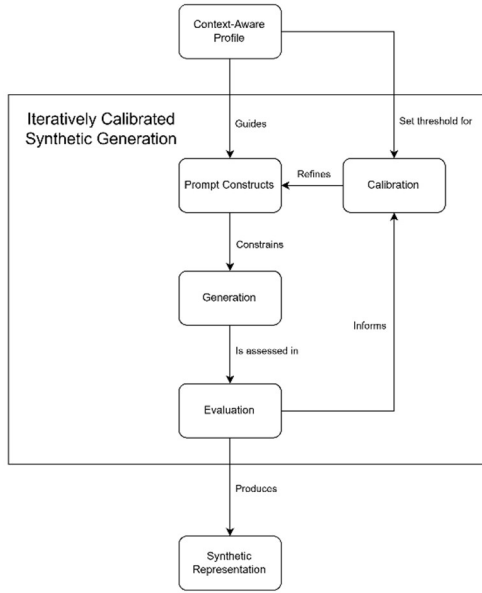


Figure 3. Iteratively Calibrated Synthetic Generation framework

The framework also extends the distributional comparison principle demonstrated by Leinonen et al. (2025). Rather than using distributional comparison only as a final evaluation, the proposed framework incorporates the observed discrepancy into the generation process itself.

2.4 Integrated Conceptual Contribution

The three conceptual artefacts form a collective structure. The context-aware characteristic classification framework defines the observable phenomena and distinguishes their occurrence according to task dependence. The Context-Aware Synthetic Representation Framework organises these classifications into a general representation comprising context-independent and context-dependent empirical profiles. The Iteratively Calibrated Synthetic Generation Framework then operationalises the target representation by translating it into generation constraints and refining those constraints according to observed discrepancies.

Ultimately, the contributions are not merely a method for prompting an LLM to produce novice-like code, but a conceptual approach in which authentic empirical characteristics determine the representation target, contextual variation is explicitly preserved, and synthetic generation is iteratively calibrated against that target while maintaining functional validity.

References

- Denzler, B., Vahid, F., Pang, A., & Salloum, M. (2024). Style Anomalies Can Suggest Cheating in CS1 Programs. *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2024*, 381–387. <https://doi.org/10.1145/3649217.3653626>
- Effenberger, T., & Pelanek, R. (2022). Code Quality Defects across Introductory Programming Topics. *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE 2022)*. <https://doi.org/10.1145/3478431.3499415>
- Fwa, H. L. (2024). Experience Report: Identifying common misconceptions and errors of novice programmers with ChatGPT. *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, ICSE-SEET '24*, 233–241. <https://doi.org/10.1145/3639474.3640059>
- Haindl, P., & Weinberger, G. (2024). Does ChatGPT Help Novice Programmers Write Better Code? Results From Static Code Analysis. *IEEE Access*, 12, 114146–114156. <https://doi.org/10.1109/ACCESS.2024.3445432>
- Leinonen, J., Denny, P., Kiljunen, O., MacNeil, S., Sarsa, S., & Hellas, A. (2025). LLM-itation is the Sincerest Form of Data: Generating Synthetic Buggy Code Submissions for Computing Education. *Proceedings of the 27th Australasian Computing Education Conference, ACE '25*, 56–63. <https://doi.org/10.1145/3716640.3716647>
- Rechackova, A., Pelanek, R., & Effenberger, T. (2024). Catalog of Code Quality Defects in Introductory Programming. *Proceedings of the 2024 Innovation and Technology in Computer Science Education Conference (ITiCSE 2024)*. <https://doi.org/10.1145/3649217.3653638>
- Rechackova, A., Pelanek, R., & Effenberger, T. (2025). Code Quality Defects in Introductory Programming. *ACM Transactions on Computing Education*, 25(4). <https://doi.org/10.1145/3749996>
- Recker, J., Indulska, M., Green, P. F., Burton-Jones, A., & Weber, R. (2019). Information systems as representations: A review of the theory and evidence. *Journal of the Association for Information Systems*, 20(6), 735–786. <https://doi.org/10.17705/1jais.00550>
- Wand, Y., & Weber, R. (1988). An Ontological Analysis of Some Fundamentals Information Systems Concepts. *Journal of the Association for Information Systems*, 35. <http://aisel.aisnet.org/icis1988/35>
- Zhang, A. G., Tang, X., Oney, S., & Chen, Y. (2024). CFlow: Supporting Semantic Flow Analysis of Students' Code in Programming Problems at Scale. *Proceedings of the Eleventh ACM Conference on Learning @ Scale, L@S '24*, 188–199. <https://doi.org/10.1145/3657604.3662025>